CENTRE IVOIRIEN DE ROBOTIQUE

CIR-DEV-WI-01 · Full Stack Web × IoT

PROJET INTEGRATEUR FINAL

TASKFLOW

Application de gestion de taches et de projets

Sections 1 à 4 · Projet individuel · 5 jours

React.js	Node.js	PostgreSQL	Redis	Docker	JWT + REST

Ce projet est la démonstration que tu peux construire une vraie application de A a Z.

1 · CONTEXTE ET OBJECTIF DU PROJET

TaskFlow est une application web de gestion de taches inspirée de Trello et Asana. Elle permet à un utilisateur de s'inscrire, de créer des projets, d'y ajouter des taches, de les organiser par statut, et de suivre sa progression en temps réel.

Ce projet n'est pas un exercice de plus — c'est une application réelle que tu pourras montrer lors d'un entretien ou ajouter à ton portfolio GitHub public. Chaque technologie apprise dans les 4 sections précédentes à un role precis dans cette application.

Pourquoi ce projet est différent des exercices précédents

Dans les sections 1 à 4, chaque exercice se concentrait sur UNE technologie.
Dans TaskFlow, tout est connecte : si tu supprimes une tache (React), l'API (Node.js) met à jour
La base de données (PostgreSQL), invalide le cache (Redis), et notifie les clients connectes (WebSocket).
Tu verras pour la première fois comment chaque outil interagit avec les autres dans une vraie application.

Les 3 grandes fonctionnalités de TaskFlow

Fonctionnalité 1 — Gestion des utilisateurs

Inscription avec nom, email, mot de passe hache (bcrypt)
Connexion avec génération d'un token JWT (24h de validite)
Profil utilisateur : modifier son nom, changer son mot de passe
Deconnexion : invalider la session dans Redis

Fonctionnalite 2 — Gestion des projets et taches

Creer, modifier et supprimer des projets
Dans chaque projet : ajouter des taches avec titre, description, priorite, date limite
4 statuts de tache : A faire → En cours → En review → Termine
Filtrer et trier les taches (par statut, priorite, date limite)
Statistiques par projet : nombre de taches par statut, taux de completion

Fonctionnalite 3 — Temps reel et performance

Notifications en temps reel via WebSocket : quand une tache change de statut
Cache Redis sur les routes frequentes : GET /projects, GET /tasks
Invalidation du cache automatique apres chaque modification

> Dashboard de synthese : mes projets, mes taches du jour, statistiques globales

2 · ARCHITECTURE TECHNIQUE

Le role de chaque technologie dans TaskFlow

Section	Technologie	Role dans TaskFlow
Section 1	HTML5 / CSS3	Structure et style de l'interface : pages login, dashboard, detail projet
Section 2	React.js	Interface dynamique : composants TaskCard, ProjectList, Dashboard
Section 2	React Router	Navigation entre pages : /, /login, /projects/:id, /dashboard
Section 2	Context API	Theme dark/light + etat de l'utilisateur connecte global
Section 2	Recharts	Graphique de completion des taches par projet
Section 3	Node.js + Express	API REST : toutes les routes CRUD pour projets et taches
Section 3	JWT	Securiser les routes : seul l'auteur peut modifier ses projets
Section 3	PostgreSQL	Stocker durablement utilisateurs, projets et taches
Section 3	Sequelize ORM	Modeliser les relations : User → Projects → Tasks
Section 4	Redis	Cache des listes + invalidation + sessions + rate limiting
Section 4	Docker	Containeriser API + PostgreSQL + Redis pour deploiement reproductible
Section 4	WebSocket	Notifier en temps reel les changements de statut des taches
Section 4	GitHub Actions	CI/CD : tests automatiques + deploiement sur chaque push main

Architecture des donnees — 3 tables liees

users → id, name, email, password, createdAt, updatedAt
|
projects → id, title, description, color, userId (FK), createdAt, updatedAt
|
tasks → id, title, description, status, priority, dueDate, projectId (FK), createdAt, updatedAt

Relations : un User a plusieurs Projects. Un Project a plusieurs Tasks.
Contrainte : un utilisateur ne voit et ne modifie QUE ses propres projets et taches.

Architecture des services — Flux d'une requete

1. L'utilisateur clique 'Terminer la tache' dans React (Section 2)

2. React envoie PATCH /api/tasks/42 { status: 'done' } avec le token JWT dans le header
3. Express recoit la requete → middleware authenticate verifie le JWT (Section 3)
4. Le controller met a jour la tache dans PostgreSQL via Sequelize (Section 3 & 4)
5. Redis invalide le cache du projet concerne (Section 4)
6. Le serveur WebSocket notifie tous les clients connectes (Section 4)
7. React met a jour l'interface sans recharger la page (Section 2)

3 · PLANNING JOUR PAR JOUR — 5 JOURS

La semaine est decoupee en 5 journees de travail. Chaque journee a un objectif clair, des taches matin et apres-midi, et un livrable concret a valider avant de passer a la suivante.

JOUR 1 — Fondations : Base de donnees + API de base

Matin

- Creer le repo GitHub 'taskflow' avec les dossiers /api et /frontend
- Initialiser le projet Node.js dans /api : npm init, installer express, sequelize, pg, bcryptjs, jsonwebtoken, dotenv
- Configurer la connexion PostgreSQL dans src/config/database.js
- Creer les 3 modeles Sequelize : User, Project, Task avec leurs associations

Apres-midi

- Ecrire les migrations Sequelize CLI pour les 3 tables
- Creer les routes d'authentification : POST /api/auth/register et POST /api/auth/login
- Tester register et login dans Postman — verifier que le token JWT est bien retourne
- Creer les routes CRUD basiques pour les projets (GET, POST, PUT, DELETE)

Livrable du jour : *API demarree sur localhost:3000. Register + login fonctionnels dans Postman. 3 tables en base.*

JOUR 2 — API complete + Cache Redis

Matin

- Finir les routes CRUD pour les taches (avec filtres par statut et priorite)
- Ajouter le middleware authenticate sur toutes les routes protegees
- Ajouter express-validator sur les routes POST et PUT
- Tester l'ensemble des routes dans Postman avec un token valide et invalide

Apres-midi

- Installer ioredis et configurer la connexion Redis
- Implementer le Cache-Aside sur GET /api/projects et GET /api/projects/:id/tasks
- Implementer l'invalidation du cache apres chaque POST/PUT/DELETE
- Ajouter le rate limiter sur POST /api/auth/login (max 5 tentatives/minute)

Livrable du jour : *API complete testee dans Postman. fromCache: true visible sur les GET repetes. 429 apres 5 tentatives login.*

JOUR 3 — Frontend React — Interface complete

Matin

- Initialiser le projet React avec Vite dans /frontend
- Installer react-router-dom, axios, recharts, framer-motion
- Creer la structure des composants : Layout, AuthGuard, TaskCard, ProjectCard

- Implementer les pages Login et Register avec formulaires controles (useState)

Apres-midi

- Creer la page Dashboard : liste des projets + statistiques globales
- Creer la page Detail Projet : liste des taches avec drag-and-drop de statut
- Implementer le Context API pour l'utilisateur connecte (token, infos profil)
- Connecter React a l'API via le service api.js (axios + intercepteur JWT)

Livrable du jour : *Application React fonctionnelle connectee a l'API. Login, dashboard et detail projet accessibles.*

JOUR 4 — Docker + WebSocket + Temps reel

Matin

- Ecrire le Dockerfile de l'API (multi-stage, node:20-alpine)
- Ecrire le docker-compose.yml avec les 4 services : api, postgres, redis, nginx
- Tester docker-compose up --build — verifier que tout demarre sans erreur
- Ajouter les variables d'environnement dans les secrets GitHub

Apres-midi

- Integrer le serveur WebSocket dans l'API Node.js
- Emettre un evenement WebSocket quand une tache change de statut
- Dans React : se connecter au WebSocket et mettre a jour l'interface en temps reel
- Tester : ouvrir deux onglets, changer le statut d'une tache, voir la mise a jour dans l'autre onglet

Livrable du jour : *docker-compose up --build lance tout. Mise a jour de statut visible en temps reel sur 2 onglets.*

JOUR 5 — CI/CD + Deploiement + Finalisation

Matin

- Ecrire les tests Jest/Supertest : au moins 6 tests couvrant les routes principales
- Creer le workflow GitHub Actions : test → build Docker → deploy
- Deployer l'API + PostgreSQL + Redis sur Railway
- Configurer les variables d'environnement de production sur Railway

Apres-midi

- Deployer le frontend React sur Vercel avec VITE_API_URL pointant vers Railway
- Tester la chaine complete en production : register → login → creer projet → ajouter tache
- Verifier le score Lighthouse (>= 85 performance, >= 90 accessibilite)
- Rediger le README.md avec URL live, screenshot, et description des fonctionnalites

Livrable du jour : *URL Railway publique + URL Vercel publique. Tests GitHub Actions verts. README complet sur GitHub.*

4 · GUIDE TECHNIQUE — STRUCTURE DU CODE

Structure des fichiers attendus

```
// Structure — Organisation du projet TaskFlow
taskflow/
  api/                          # Backend Node.js
    src/
      config/
        database.js             # Connexion Sequelize + PostgreSQL
        redis.js                # Connexion ioredis
        swagger.js              # Documentation API
      models/
        User.js                 # Modele utilisateur
        Project.js              # Modele projet
        Task.js                 # Modele tache
      migrations/               # Fichiers de migration Sequelize CLI
      routes/
        auth.js                 # POST /register, POST /login
        projects.js             # CRUD /projects
        tasks.js                # CRUD /projects/:id/tasks
      controllers/
        authController.js
        projectsController.js
        tasksController.js
      middleware/
        authenticate.js         # Verifier le JWT
        cache.js                # Cache-Aside Redis
        rateLimiter.js          # Rate limiting
        errorHandler.js         # Gestionnaire d'erreurs centralise
      websocket/
        ws.js                   # Serveur WebSocket + emission
d'evenements
      app.js                    # Configuration Express
      server.js                 # Point d'entree + sync Sequelize
    tests/
      auth.test.js
      projects.test.js
      tasks.test.js
    Dockerfile
    docker-compose.yml
    .env.example
    .gitignore
    package.json

  frontend/                     # Frontend React
    src/
      components/
        TaskCard/               # Carte d'une tache avec changement de
statut
        ProjectCard/            # Carte d'un projet avec progression
        Layout/                 # Header + Sidebar + Outlet
        AuthGuard/              # Rediriger si non connecte
        StatusBadge/            # Badge colore selon le statut
```

```
      pages/
        Login.jsx               # Formulaire connexion
        Register.jsx            # Formulaire inscription
        Dashboard.jsx           # Vue d'ensemble + statistiques
        Projects.jsx            # Liste des projets
        ProjectDetail.jsx       # Taches d'un projet
        Profile.jsx             # Modifier son profil
      context/
        AuthContext.jsx         # Utilisateur connecte + token
        ThemeContext.jsx        # Dark/Light mode
      services/
        api.js                  # Axios + intercepteur JWT
        websocket.js            # Connexion WebSocket
      hooks/
        useProjects.js          # Hook custom : charger les projets
        useTasks.js             # Hook custom : charger les taches
    .env
    vite.config.js

  .github/
    workflows/
      deploy.yml               # CI/CD GitHub Actions

  README.md
```

Modeles de donnees a implementer

```
// JavaScript — Les 3 modeles Sequelize
// models/User.js
const User = sequelize.define('User', {
  id:       { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true },
  name:     { type: DataTypes.STRING(60),  allowNull: false },
  email:    { type: DataTypes.STRING(120), allowNull: false, unique: true,
validate: { isEmail: true } },
  password: { type: DataTypes.STRING,      allowNull: false },
  avatar:   { type: DataTypes.STRING,      defaultValue: null },  // URL
optionnelle
}, { tableName: 'users', timestamps: true });

// models/Project.js
const Project = sequelize.define('Project', {
  id:          { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true },
  title:       { type: DataTypes.STRING(100), allowNull: false },
  description: { type: DataTypes.TEXT,        defaultValue: '' },
  color:       { type: DataTypes.STRING(7),   defaultValue: '#2E86C1' },
// Couleur hex
  userId:      { type: DataTypes.INTEGER,     allowNull: false },
}, { tableName: 'projects', timestamps: true });

// models/Task.js
const Task = sequelize.define('Task', {
  id:          { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true },
```

```
  title:       { type: DataTypes.STRING(150), allowNull: false },
  description: { type: DataTypes.TEXT,        defaultValue: '' },
  status: {
    type:         DataTypes.ENUM('todo', 'in_progress', 'review', 'done'),
    defaultValue: 'todo',
  },
  priority: {
    type:         DataTypes.ENUM('low', 'medium', 'high', 'urgent'),
    defaultValue: 'medium',
  },
  dueDate:   { type: DataTypes.DATEONLY, defaultValue: null },
  projectId: { type: DataTypes.INTEGER, allowNull: false },
}, { tableName: 'tasks', timestamps: true });

// Associations dans server.js
User.hasMany(Project,    { foreignKey: 'userId',    as: 'projects',
onDelete: 'CASCADE' });
Project.belongsTo(User,  { foreignKey: 'userId',    as: 'owner' });
Project.hasMany(Task,    { foreignKey: 'projectId', as: 'tasks',
onDelete: 'CASCADE' });
Task.belongsTo(Project,  { foreignKey: 'projectId', as: 'project' });
```

Routes API a implementer

Liste complete des routes REST

- POST /api/auth/register — Creer un compte (Public)
- POST /api/auth/login — Se connecter, recevoir JWT (Public)
- GET /api/auth/me — Infos utilisateur connecte (JWT)
- PUT /api/auth/profile — Modifier nom/mot de passe (JWT)
- GET /api/projects — Tous mes projets + cache Redis (JWT)
- POST /api/projects — Creer un projet (JWT)
- GET /api/projects/:id — Detail d'un projet (JWT)
- PUT /api/projects/:id — Modifier un projet (JWT)
- DELETE /api/projects/:id — Supprimer projet + taches (JWT)
- GET /api/projects/:id/tasks — Taches filtrables + cache (JWT)
- POST /api/projects/:id/tasks — Creer une tache (JWT)
- PATCH /api/tasks/:id/status — Changer le statut + WebSocket (JWT)
- PUT /api/tasks/:id — Modifier une tache (JWT)
- DELETE /api/tasks/:id — Supprimer une tache (JWT)
- GET /api/dashboard — Stats globales cache 30s (JWT)
- GET /api/health — Health check Railway (Public)
- GET /api-docs — Documentation Swagger interactive (Public)

Exemples de code cles

Route PATCH /tasks/:id/status avec WebSocket

```
// JavaScript — Controller avec emission WebSocket
// controllers/tasksController.js
const updateStatus = async (req, res, next) => {
  try {
    const { status } = req.body;
    const task = await Task.findOne({
      where: { id: req.params.id },
      include: [{ model: Project, as: 'project' }]
    });

    if (!task) return res.status(404).json({ success: false, error: 'Tache
non trouvee' });
    if (task.project.userId !== req.user.id)
      return res.status(403).json({ success: false, error: 'Acces refuse'
});

    const oldStatus = task.status;
    await task.update({ status });

    // Invalider le cache du projet
    await redis.del(`cache:${req.user.id}:/projects/${task.projectId}`);
    await
redis.del(`cache:${req.user.id}:/projects/${task.projectId}/tasks`);

    // Emettre l'evenement WebSocket aux clients connectes
    broadcast(req.user.id, {
      type     : 'TASK_STATUS_CHANGED',
      taskId   : task.id,
      oldStatus,
      newStatus: status,
      projectId: task.projectId,
    });

    res.json({ success: true, data: task });
  } catch (err) { next(err); }
};
```

Composant React TaskCard avec mise a jour optimiste

```
// JSX — TaskCard avec changement de statut
// components/TaskCard/TaskCard.jsx
import { useState } from 'react';
import { motion  } from 'framer-motion';
import { sensorsApi } from '../../services/api';

const STATUS_LABELS = {
  todo        : 'A faire',
  in_progress : 'En cours',
  review      : 'En review',
  done        : 'Termine',
};

const STATUS_COLORS = {
  todo        : '#888888',
  in_progress : '#2E86C1',
```

```
  review      : '#E67E22',
  done        : '#1E8449',
};

const TaskCard = ({ task, onStatusChange }) => {
  const [isUpdating, setIsUpdating] = useState(false);

  const handleStatusChange = async (newStatus) => {
    setIsUpdating(true);
    try {
      await api.patch(`/tasks/${task.id}/status`, { status: newStatus });
      onStatusChange(task.id, newStatus);  // Mise a jour locale immediate
    } catch (err) {
      console.error('Erreur mise a jour statut:', err);
    } finally {
      setIsUpdating(false);
    }
  };

  return (
    <motion.article
      layout
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, scale: 0.95 }}
      className='task-card'
    >
      <h3>{task.title}</h3>
      <p>{task.description}</p>
      <select
        value={task.status}
        onChange={e => handleStatusChange(e.target.value)}
        disabled={isUpdating}
        style={{ color: STATUS_COLORS[task.status] }}
      >
        {Object.entries(STATUS_LABELS).map(([value, label]) => (
          <option key={value} value={value}>{label}</option>
        ))}
      </select>
    </motion.article>
  );
};
```

Hook WebSocket dans React

```
// JavaScript — hooks/useWebSocket.js
import { useEffect, useRef } from 'react';

const useWebSocket = (onMessage) => {
  const wsRef = useRef(null);

  useEffect(() => {
    const token = localStorage.getItem('jwt_token');
    if (!token) return;
```

```
    const wsUrl = import.meta.env.VITE_WS_URL || 'ws://localhost:3000';
    wsRef.current = new WebSocket(`${wsUrl}?token=${token}`);

    wsRef.current.onopen    = () => console.log('WebSocket connecte');
    wsRef.current.onmessage = (event) => {
      const message = JSON.parse(event.data);
      onMessage(message);
    };
    wsRef.current.onclose   = () => console.log('WebSocket ferme');

    return () => wsRef.current?.close();
  }, []);
};

// Utilisation dans ProjectDetail.jsx :
useWebSocket((message) => {
  if (message.type === 'TASK_STATUS_CHANGED') {
    setTasks(prev => prev.map(t =>
      t.id === message.taskId ? { ...t, status: message.newStatus } : t
    ));
  }
});
```

5 · GRILLE D'EVALUATION — 100 POINTS

La note finale est sur 100 points. Chaque critere evalue non pas si ca fonctionne, mais si c'est fait de la bonne facon, en respectant les bonnes pratiques vues en cours.

Critere d'evaluation	Ce qui est verifie	Points
SECTION 1 & 2 — Frontend React		
Interface React structuree	Composants decoupes, props propres, pas de logique dans JSX	**10 pts**
Navigation React Router	4 pages fonctionnelles, AuthGuard sur les routes privees	**8 pts**
Formulaires controles	Login, Register, Ajout tache avec validation useState	**7 pts**
Context API (auth + theme)	Token et user disponibles partout sans prop drilling	**5 pts**
Graphique Recharts	Statistiques de completion visibles sur le dashboard	**5 pts**
Animations Framer Motion	Minimum : entree des TaskCards et transitions de pages	**5 pts**
SECTION 3 — Backend Node.js + PostgreSQL		
API REST complete	17 routes implementees, codes HTTP corrects (201, 400, 401, 403, 404)	**10 pts**
Authentification JWT	Register + login + middleware authenticate + routes protegees	**8 pts**
Modeles Sequelize + migrations	3 modeles avec associations, migrations CLI (pas sync())	**7 pts**
Validation express-validator	Toutes les routes POST/PUT validees avec messages d'erreur	**5 pts**
Documentation Swagger	Au moins 10 routes documentees, testables depuis /api-docs	**5 pts**
SECTION 4 — Redis + Docker + CI/CD + WebSocket		
Cache Redis fonctionnel	GET projects/tasks: fromCache:true au 2eme	**8 pts**

	appel, invalidation apres modif	
Mise a jour en temps reel	Changement de statut visible sans recharger dans un 2eme onglet	**7 pts**
docker-compose.yml complet	docker-compose up --build demarre tout en moins de 3 minutes	**5 pts**
GitHub Actions (CI/CD)	Tests verts + deploiement automatique sur push main	**5 pts**
PRODUCTION & QUALITE		
Application deployee en production	URL Railway (API) + URL Vercel (frontend) accessibles	**5 pts**
README.md complet	URL live, screenshot, description fonctionnalites, instructions install	**5 pts**
TOTAL	**100 pts**	

Seuils et badges

0 – 49 pts Insuffisant — Recommencer les bases (Sections 1 & 2)
50 – 64 pts Passable — Fonctionnel mais lacunes importantes sur le backend
65 – 79 pts Bien — Application fonctionnelle avec quelques manques sur Section 4
80 – 89 pts Tres bien — Proche d'une application professionnelle
90 – 100 pts Excellent — Badge 'TaskFlow Developer' + mention sur le portfolio CIR

6 · LIVRABLE — CE QUI EST SOUMIS SUR MOODLE

Checklist avant soumission

- Lien GitHub du repo 'taskflow' (public ou prive avec acces formateur)
- URL de l'API Railway : https://taskflow-api.railway.app
- URL du frontend Vercel : https://taskflow.vercel.app
- Actions GitHub : tous les jobs en vert (test + deploy)
- README.md avec screenshot du dashboard, URL live, instructions d'installation
- Fichier .env.example (les cles reelles ne doivent PAS etre dans le repo)

Ce qui ne doit JAMAIS etre dans le repo GitHub

Le fichier .env (mots de passe, JWT_SECRET, DATABASE_URL)
Le dossier node_modules/ (genere par npm install)
Les fichiers de logs (.log)
Les fichiers de build du frontend (/frontend/dist/)

Format du README.md attendu

```
# TaskFlow — Application de gestion de taches

## Demo en ligne
Frontend : https://taskflow.vercel.app
API : https://taskflow-api.railway.app/api-docs

## Fonctionnalites
- Authentification JWT (register / login)
- Gestion de projets et de taches (CRUD complet)
- Mise a jour du statut en temps reel (WebSocket)
- Cache Redis pour les performances
- Interface React avec theme Dark/Light

## Stack technique
Frontend : React.js, React Router, Context API, Recharts, Framer Motion, Vercel
Backend : Node.js, Express, JWT, Sequelize, PostgreSQL, Redis, WebSocket, Railway
DevOps : Docker, docker-compose, GitHub Actions CI/CD

## Installation locale
git clone https://github.com/username/taskflow
cd taskflow && cp api/.env.example api/.env
# Remplir les variables dans api/.env
docker-compose up --build
```

7 · CONSEILS POUR REUSSIR

Les 5 regles d'or du projet

Regle 1 — Commencer par l'API, pas par React

Beaucoup d'apprenants commencent par le frontend car c'est plus visuel.
C'est une erreur : React ne peut rien faire si l'API n'existe pas.
Suivre le planning : Jours 1-2 = API complete et testee dans Postman.
Seulement quand toutes les routes repondent correctement, commencer React (Jour 3).

Regle 2 — Tester chaque etape dans Postman avant d'avancer

Ne pas passer au controleur suivant si le precedent n'est pas valide.
Chaque route doit etre testee avec un token valide ET sans token (pour verifier le 401).
Sauvegarder une collection Postman avec toutes les requetes — elle sera soumise avec le projet.

Regle 3 — Commiter souvent avec des messages clairs

Un commit par fonctionnalite terminee, pas un seul commit a la fin du projet.
Format des messages : 'feat: ajouter route PATCH /tasks/:id/status'
Format des messages : 'fix: corriger le middleware authenticate sur les routes tasks'
L'historique Git sera evalue — un seul commit enorme est un signal negatif.

Regle 4 — Ne pas copier-coller sans comprendre

Les exemples de code dans ce document sont des guides, pas des solutions completes.
Adapter chaque exemple a ton projet. Si tu copies sans comprendre, tu seras bloque des que quelque chose ne fonctionne pas.
Si tu bloques plus de 30 minutes sur un probleme : poser la question.

Regle 5 — La fonctionnalite prime sur l'esthetique

Un dashboard qui marche avec un CSS simple vaut plus qu'un beau dashboard qui plante.
Prioriser : auth fonctionnelle → CRUD projets/taches → cache Redis → WebSocket → CI/CD
Le design et les animations Framer Motion sont les derniers points a ajouter.
Si tu manques de temps en Jour 5 : un Lighthouse < 85 est moins grave qu'un deploy qui plante.

Les erreurs les plus frequentes

Erreur : CORS bloque React en production

Cause : FRONTEND_URL dans Railway ne correspond pas exactement a l'URL Vercel.
Solution : verifier que FRONTEND_URL = https://taskflow.vercel.app (sans slash final).
En local : ajouter aussi http://localhost:5173 dans les origines CORS autorisees.

Erreur : Le WebSocket ne se connecte pas en production

Cause : l'URL WebSocket doit utiliser wss:// en production (pas ws://).
Solution : VITE_WS_URL = wss://taskflow-api.railway.app dans les variables Vercel.
Railway gere le WSS automatiquement — pas besoin de configuration supplementaire.

Erreur : Les donnees ne s'affichent pas apres login

Cause la plus frequente : l'intercepteur axios ne trouve pas le token dans localStorage.
Verifier : localStorage.getItem('jwt_token') dans la console du navigateur apres login.
Si vide : s'assurer que le login sauvegarde bien le token : localStorage.setItem('jwt_token', data.token).